

Servicio Nacional de Aprendizaje SENA

Regional Cauca C.T.P.I

Centro de Teleinformática y Producción Industrial

Competencia

Ejercicios POO – Herencia Polimorfismo Encapsulación

Aprendiz

Lucero Valencia Bohórquez

Instructor

Yoly Alexandra Muñoz Guevara

Programa de formación

Analisis y Desarrollo de Software

FICHA: 3407734

2026

Contenido

1. Introducción 3

2. Objetivo general 3

3. Objetivos específicos 3

4. Descripción del sistema 3

5. Aplicación de los principios de POO 4

5.1. Clases y objetos 4

5.2. Herencia 4

5.3. Encapsulamiento 4

5.4. Polimorfismo 5

6. Validación y control de errores 5

7. Importancia de las validaciones 5

8. Conclusiones 6

TALLER POO JAVASCRIPT 10 EJERCICIOS

EJERCICIO 1. SISTEMA DE CUENTAS BANCARIAS

Programa de formación: Análisis y Desarrollo de Software

Tema: Programa Orientada a Objetos (POO) en JavaScript

Actividad: Diseño de un sistema para administrar cuentas bancarias.

1. Introducción

En este taller se plantea el desarrollo de un sistema de cuentas bancarias utilizando JavaScript y los principios de la Programación Orientada a Objetos (POO). El propósito es representar diferentes tipos de cuentas, administrar sus saldos y establecer operaciones para consignar, retirar y consultar dinero.

Durante el desarrollo se utilizan conceptos como clases, objetos, herencia, encapsulamiento y polimorfismo. También se consideran validaciones para prevenir operaciones incorrectas, como consignaciones negativas, retiros superiores al saldo disponible y consultas de cuentas inexistentes.

2. Objetivo general

Diseñar un sistema de cuentas bancarias mediante JavaScript, aplicando los principios de la Programación Orientada a Objetos para organizar el código, reutilizar métodos y controlar las operaciones financieras.

3. Objetivos específicos

- Crear una clase principal denominada CuentaBancaria con las propiedades y los métodos comunes.
- Implementar las clases derivadas CuentaAhorros, CuentaCorriente y CuentaNomina.
- Aplicar la herencia para reutilizar propiedades y métodos.
- Utilizar el encapsulamiento para proteger información importante, como el número de cuenta y el saldo.
- Aplicar el polimorfismo para establecer reglas de retiro diferentes según el tipo de cuenta.
- Validar los datos ingresados y controlar los errores que puedan presentarse durante las operaciones.

4. Descripción del sistema

El sistema permite administrar diferentes cuentas bancarias. Cada cuenta contiene un número identificador, el nombre del titular y un saldo inicial.

Se establece una clase padre llamada CuentaBancaria, de la cual heredan las clases CuentaAhorros, CuentaCorriente y CuentaNomina.

Las operaciones principales son:

- **Consignar:** permite agregar dinero a una cuenta, siempre que la cantidad sea mayor que cero.
- **Consultar saldo:** permite conocer el saldo actual de la cuenta.
- **Retirar:** permite retirar dinero, verificando que la cantidad sea válida y que se cumplan las reglas del tipo de cuenta.

Las cuentas se almacenarán en un arreglo para facilitar su administración, búsqueda y consulta.

5. Aplicación de los principios de POO

5.1. Clases y objetos

Una clase es una estructura que define las características y los comportamientos de un elemento. En este ejercicio, CuentaBancaria representa el modelo general de una cuenta.

Un objeto es una instancia de una clase. Por ejemplo, se puede crear una cuenta de ahorros con un número, un titular y un saldo determinados.

5.2. Herencia

La herencia permite que una clase hija reutilice las características y los métodos de una clase padre.

En este ejercicio, las clases CuentaAhorros, CuentaCorriente y CuentaNomina heredan de CuentaBancaria. Por ello, pueden utilizar los métodos consignar() y consultarSaldo() sin necesidad de escribirlos nuevamente.

La palabra reservada extends establece la herencia, mientras que super() permite llamar al constructor de la clase padre para inicializar los datos de la cuenta.

5.3. Encapsulamiento

El encapsulamiento consiste en proteger los datos internos de una clase y controlar la manera en que se accede a ellos.

En JavaScript se utilizan propiedades privadas mediante el símbolo #. En este ejercicio, el número de cuenta y el saldo se declaran como propiedades privadas, evitando que se modifiquen directamente desde fuera de la clase.

Por ejemplo:

- #numero: almacena el número de la cuenta de manera privada.
- #saldo: almacena el saldo y evita su modificación directa desde el exterior.
- titular: almacena el nombre del titular como una propiedad pública.

El saldo se consulta mediante consultarSaldo() y se modifica a través de operaciones controladas como consignar() y retirar().

5.4. Polimorfismo

El polimorfismo permite que un mismo método tenga comportamientos diferentes según el objeto que lo utilice.

En este sistema, se puede aplicar al retiro de dinero. La cuenta de ahorros puede impedir los retiros superiores al saldo disponible, la cuenta corriente puede permitir un sobregiro limitado y la cuenta de nómina puede exigir que exista saldo suficiente.

De esta manera, cada tipo de cuenta puede aplicar sus propias reglas sin perder las características comunes heredadas de CuentaBancaria.

6. Validación y control de errores

El sistema debe controlar las siguientes situaciones:

Consignaciones iguales a cero o negativas: se rechazan porque no representan un ingreso válido de dinero. Se utiliza una condición para verificar que la cantidad sea mayor que cero.

Retiros iguales a cero o negativos: se rechazan porque una operación de retiro debe tener una cantidad positiva.

Retiros superiores al saldo permitido: se validan antes de descontar el dinero. En las cuentas corrientes, la validación debe considerar el límite de sobregiro establecido.

Cuentas repetidas: antes de registrar una cuenta nueva, se debe comprobar que su número no exista en el arreglo. Esto evita identificar dos cuentas diferentes con el mismo número.

Consultas de cuentas inexistentes: cuando se busca una cuenta por su número, el sistema debe comprobar que exista. Si no se encuentra, debe informar que la cuenta no está registrada.

Para generar mensajes de error se utiliza throw new Error("mensaje"). Esto permite detener una operación inválida y comunicar la causa del problema.

7. Importancia de las validaciones

Las validaciones son fundamentales porque ayudan a mantener la integridad de la información financiera y evitan resultados incorrectos.

Sin estas comprobaciones, el sistema podría aceptar cantidades negativas, permitir retiros no autorizados, registrar cuentas duplicadas o intentar consultar información inexistente.

Además, las validaciones facilitan la detección de errores y hacen que el programa sea más confiable, organizado y seguro.

8. Conclusiones

El desarrollo de este ejercicio permite comprender cómo aplicar la Programación Orientada a Objetos en una situación práctica relacionada con la administración bancaria.

La herencia facilita la reutilización de código, el encapsulamiento protege los datos de las cuentas y el polimorfismo permite establecer diferentes comportamientos para las operaciones de retiro.

Asimismo, las validaciones de errores son necesarias para controlar las operaciones y mantener la consistencia de la información.

En conclusión, la aplicación de estos principios contribuye a construir programas más organizados, fáciles de mantener y preparados para incorporar nuevas funcionalidades.